

GSAS @NIDA : Deep Learning

CNN 1

Assoc.Prof.Thitirat Siriborvornratanakul, Ph.D.

Email: thitirat@as.nida.ac.th
Website: <http://as.nida.ac.th/~thitirat/>

1

DL models in this course

~~DISCRIMINATIVE~~ **FUNDAMENTAL**

MLP

CNN

RNN

LSTM

GRU

Transformer

GENERATIVE

VAE

GAN

Diffusion model

2

OUTLINE

01 Preliminary

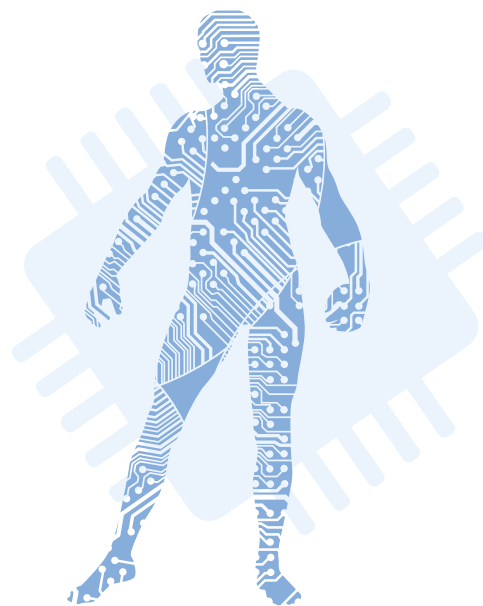
The brief history of CNN and the ImageNet moment in computer vision

02 Intro to CNNs

Learn components that make up a basic Convolutional Neural Network for image classification

03 Coding

Combine everything and implement a program with Keras



3



Preliminary

The brief history of CNN and the ImageNet moment in computer vision

4

THE OLD HISTORY

- **1943:** The first **neural network** was proposed by McCulloch and Pitts.
- **1955:** John McCarthy first coined the term **Artificial Intelligence**.
- **1959:** Samuel coined and popularized the term **Machine Learning**.

- **1986:** Birth of **backpropagation**
- **1986:** Birth of **RNN** and **AE**
- **1989:** Birth of **CNN**
- **1997:** Birth of **LSTM**

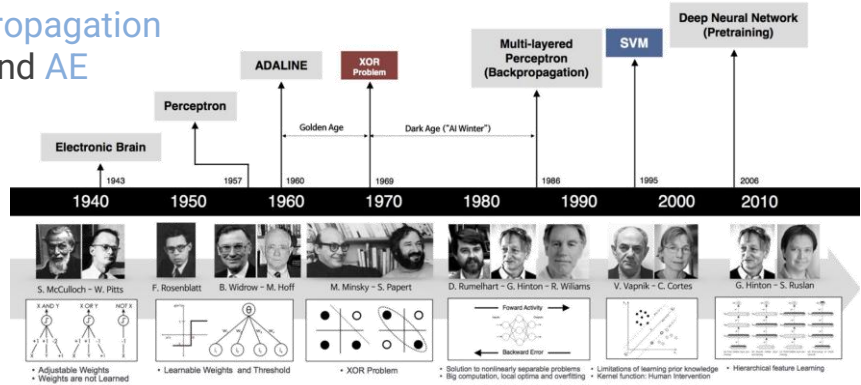


Image credit:
https://beamandrew.github.io/deeplearning/2017/02/23/deep_learning_101_part1.html

5

THE MODERN HISTORY

- **2006:** The **pretraining** technique by Hinton et al.



- **2012:** ImageNet evolution—**AlexNet**



- **2014:** Birth of **GRU**, **VAE**, and **GAN**
- **2015:** Birth of **diffusion model**

- **2017:** The **DeepFake** viral
- **2017:** Birth of the groundbreaking **Transformer**



- **2018:** NLP's ImageNet moment—**BERT**
- **2018:** ACM Turing Award to Deep NN



6

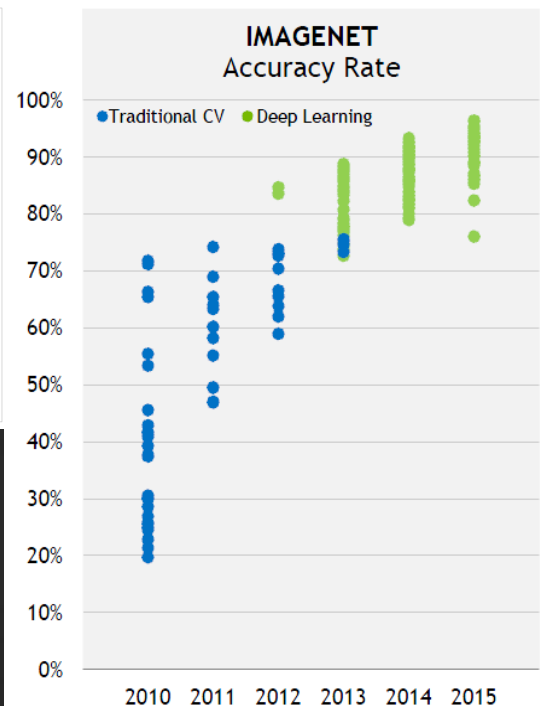
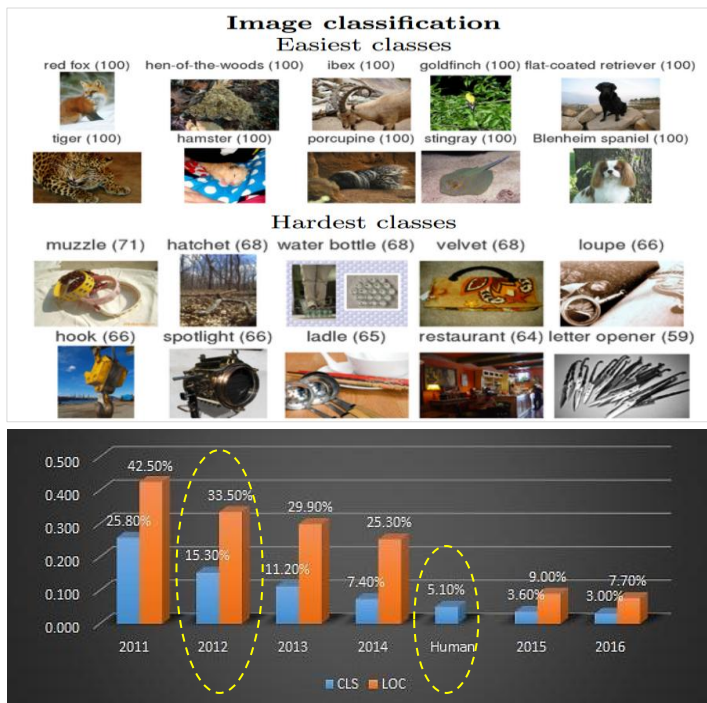
ImageNet (2010-2017)



- One of the most prestigious benchmarks in computer vision. Sometimes even referred to as the Olympics of Computer Vision.
- The **ImageNet** dataset was first presented in CVPR 2009.
- The annual **ImageNet Large Scale Visual Recognition Challenge (ILSVRC)**:
 - Started in 2010, it is designed to follow on from a similar project called PASCAL VOC which ran from 2005 until 2012.
 - 1,000 different categories of objects where contestants have to scour a database of over 1 million images to find every instance of each object.
 - Announced on Jul 26, 2017: "We are passing the baton to Kaggle (owned by Google). From now on, all three challenges (LOC-CLS, DET, VID) will be hosted on Kaggle!"

J. Deng, W. Dong, R. Socher, L. Li, K. Li, and L. Fei-Fei, "ImageNet: A large-scale hierarchical image database," In Proc. of IEEE CVPR, 2009 <https://www.explore.ieee.org/document/5206848>
O. Russakovsky, J. Deng, H. Su, J. Krause, S. Satheesh, S. Ma, Z. Huang, A. Karpathy, A. Khosla, M. Bernstein, A.C. Berg, and L. Fei-Fei, "ImageNet Large Scale Visual Recognition Challenge," In IJCV, 115, 2015 <https://arxiv.org/abs/1409.0575>

7



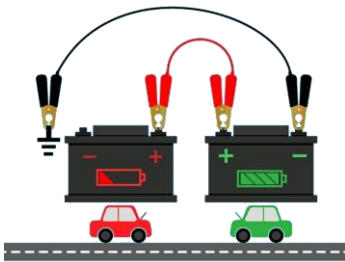
8

AI ImageNet (2010-2017)

WORLD'S STANDARD IMAGE BENCHMARK

DATASET USED BY MOST FOUNDATION MODELS

- In 2011 and 2012, **ImageNet** dataset became a benchmark for how well image classification algorithms fared against the most complex visual dataset assembled at the time.
- Researchers began to notice something more going on than just a competition—their algorithms worked better when they trained using the **ImageNet** dataset.



“The nice surprise was that people who trained their models on ImageNet could use them to jumpstart models for other recognition tasks. You’d start with the ImageNet model and then you’d fine-tune it for another task,” said Berg. “That was a breakthrough both for neural nets and just for recognition in general.”

<https://qz.com/1034972/the-data-that-changed-the-direction-of-ai-research-and-possibly-the-world/>

9



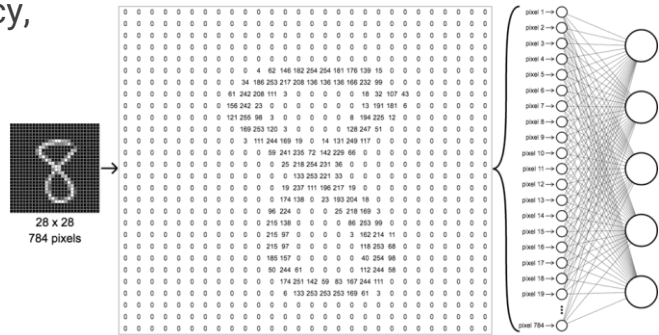
Intro to CNNs

Learn components that make up a basic Convolutional Neural Network for image classification

10

MLP for image data

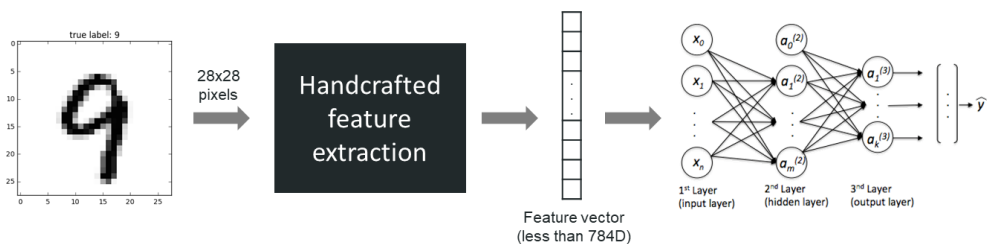
- If one input node (neuron) corresponds to one input pixel:
 - The amount of weight parameters rapidly becomes unmanageable for large images. Also, no weights are shared among pixels.
 - MLP uses fully-connected layers (Dense) that form a very dense web—resulting in redundancy, inefficiency, and overfitting.
 - MLP is not translation invariant as it reacts differently to an input image and its shifted versions.



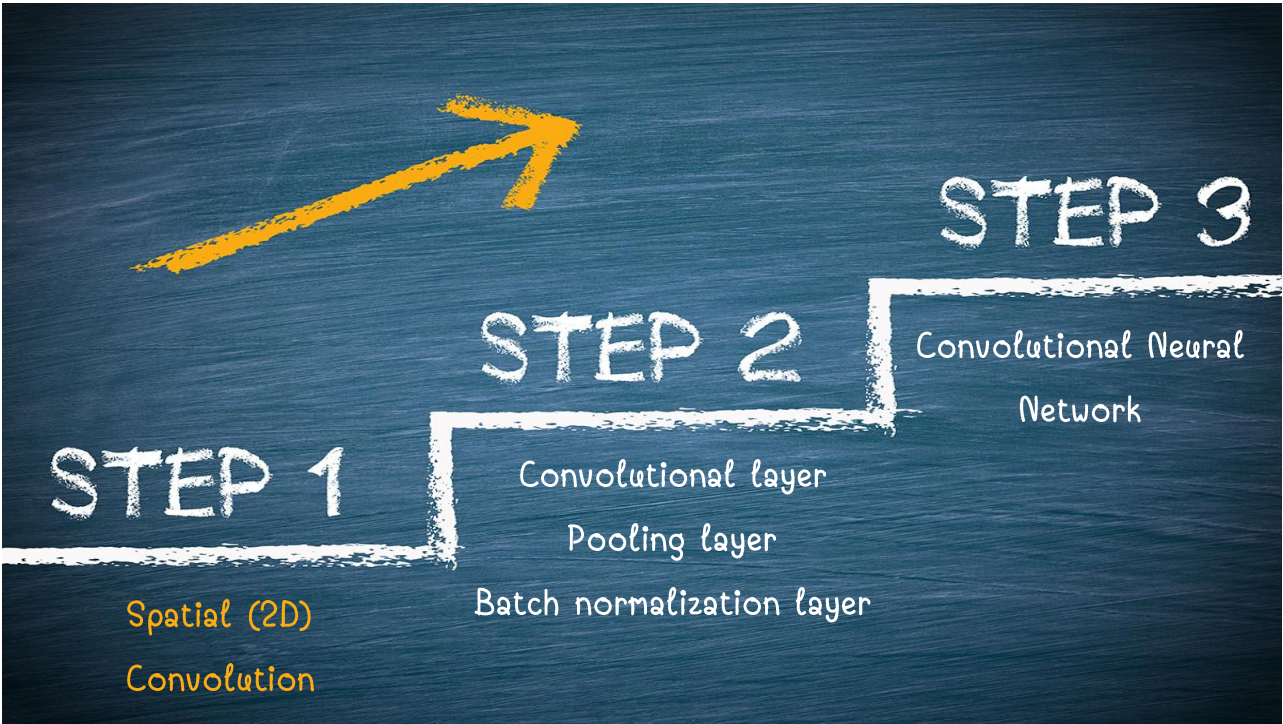
11

MLP for image data

- If an input image is first compressed using any handcrafted feature engineering technique:
 - It becomes a two-step network that is more difficult to debug and cannot be trained in an end-to-end manner using backpropagation.
 - Handcrafted feature engineering was a long-standing bottleneck in computer vision for almost half a century.



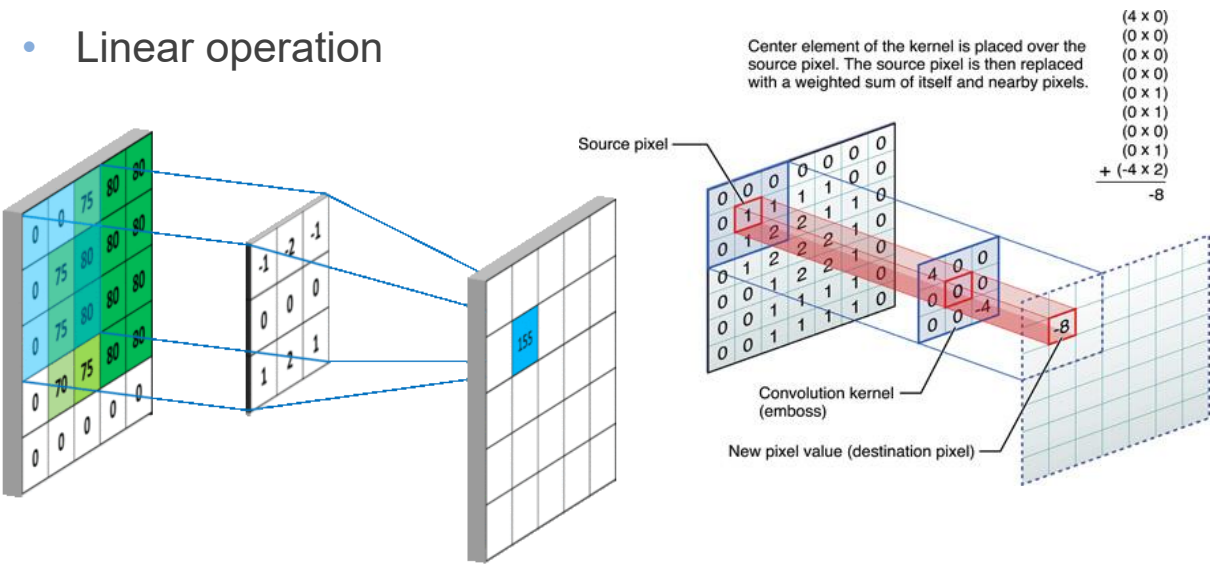
12



13

A Spatial (2D) Convolution

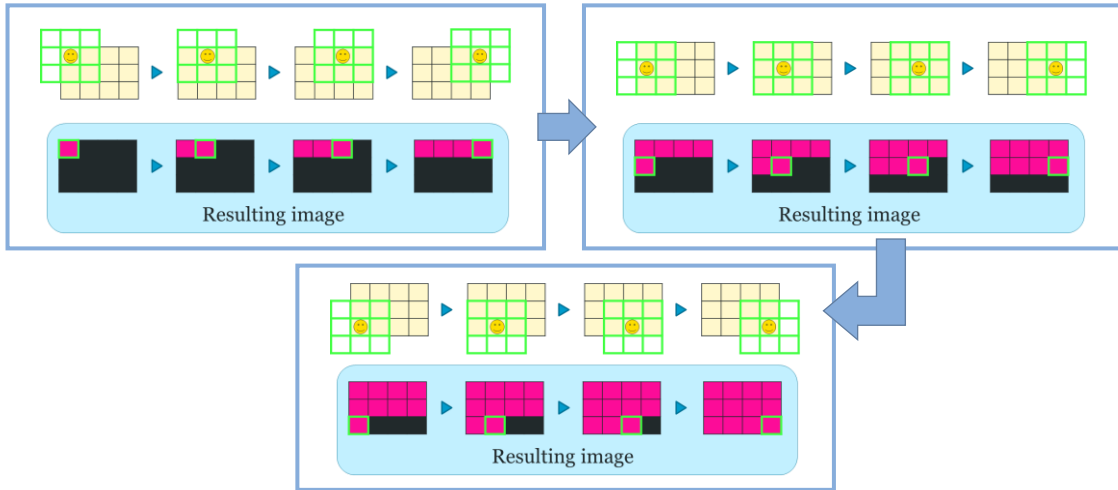
- Linear operation



14

A Spatial (2D) Convolution

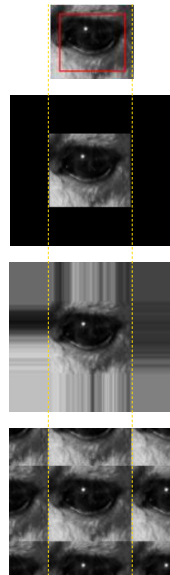
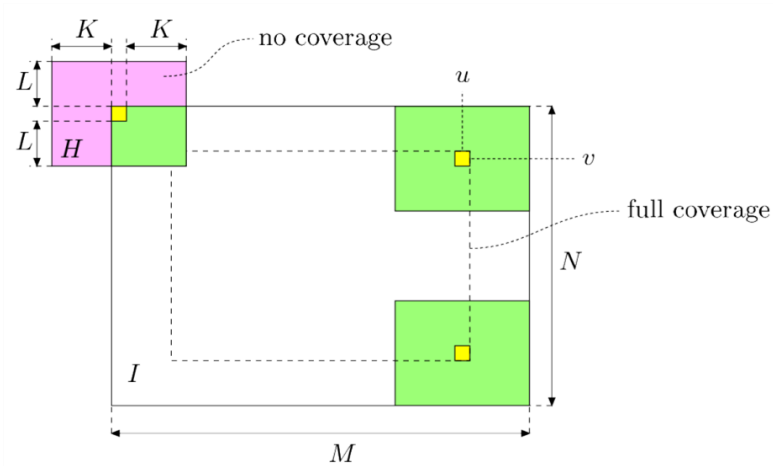
- Kernel sliding (kernel's size, stride)



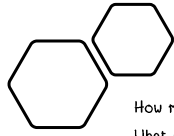
15

A Spatial (2D) Convolution

- Padding strategy: pad it or crop it



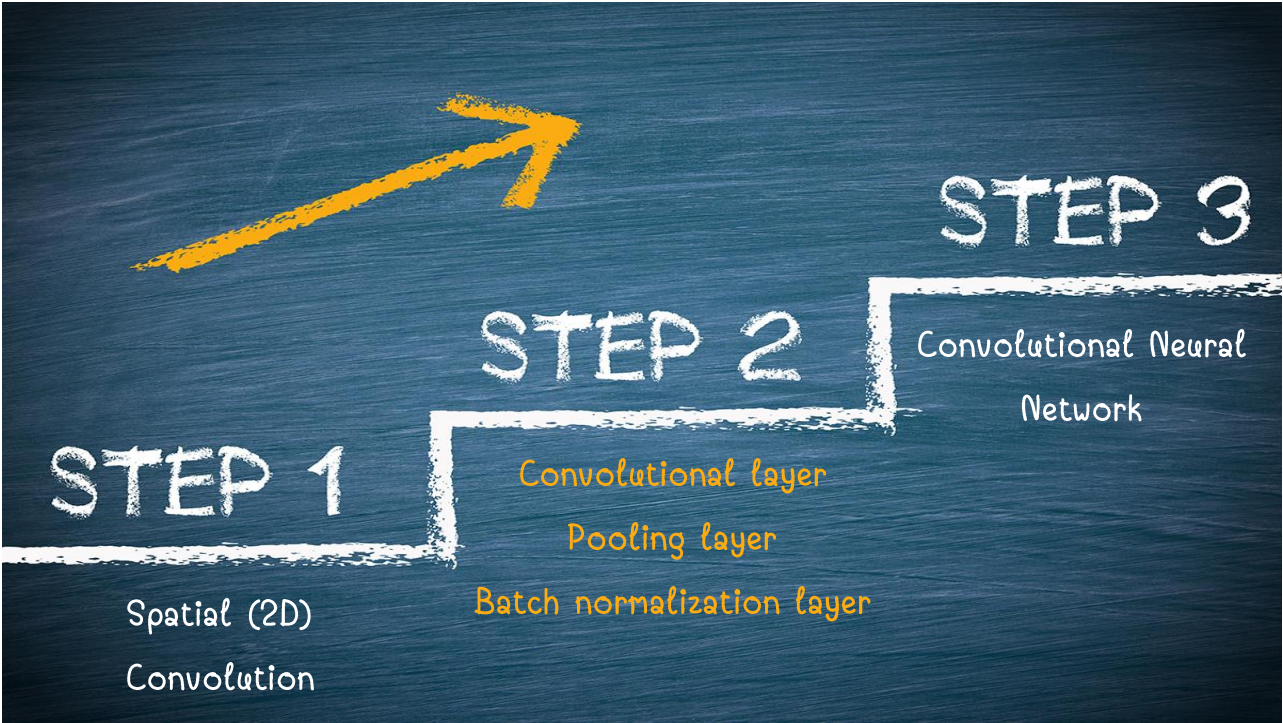
16



- How many filters?
- What filters?
- The sequence of filters?
- Kernel's size for each?
- Padding or not?
- Stride values for each?

Paradox of Choices

Infinite combinations of filters



CONV2D LAYER

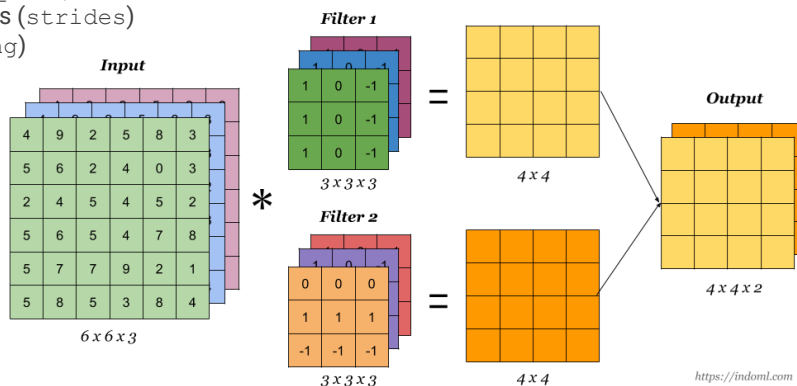
- For a single layer of `keras.layers.Conv2D` on a 3-channel input image:

- Trainable parameters:
 - Weights (numeric values in each kernel) and biases

- Hyperparameters:

These are specific to a conv layer.

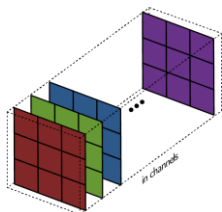
- The number of kernels (filters)
- Kernel's 2D size (`kernel_size`)
- A 2D tuple of striding steps (`strides`)
- Padding strategy (`padding`)
- `dilation_rate`
- `activation`
- `kernel_initializer`
- `kernel_regularizer`
- etc.



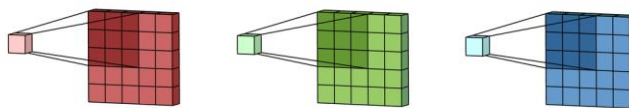
<https://indoml.com>

19

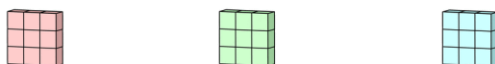
- For each filter, prepare the 2D kernel(s). The number of 2D kernels is automatically assigned by deep learning frameworks.



- Separately apply each kernel in the filter to its corresponding feature map.



- Aggregate (Sum) results from all kernels in the filter.



- Finally, add the bias.



CONV2D LAYER

The multi-channel version



Animation credit: <https://towardsdatascience.com/intuitively-understanding-convolutions-for-deep-learning-1f6f42faee1>

20

K CODE REVIEW

```
keras.layers.Conv2D(
    filters,
    kernel_size,
    strides=(1, 1),
    padding="valid",
    data_format=None,
    dilation_rate=(1, 1),
    groups=1,
    activation=None,
    use_bias=True,
    kernel_initializer="glorot_uniform",
    bias_initializer="zeros",
    kernel_regularizer=None,
    bias_regularizer=None,
    activity_regularizer=None,
    kernel_constraint=None,
    bias_constraint=None,
    **kwargs
)
```

Input shape

4-D tensor with shape: $\text{batch_shape} + (\text{channels}, \text{rows}, \text{cols})$ if `data_format='channels_first'` or 4-D tensor with shape: $\text{batch_shape} + (\text{rows}, \text{cols}, \text{channels})$ if `data_format='channels_last'`.

Output shape

4-D tensor with shape: $\text{batch_shape} + (\text{filters}, \text{new_rows}, \text{new_cols})$ if `data_format='channels_first'` or 4-D tensor with shape: $\text{batch_shape} + (\text{new_rows}, \text{new_cols}, \text{filters})$ if `data_format='channels_last'`. rows and cols values might have changed due to padding.

Returns

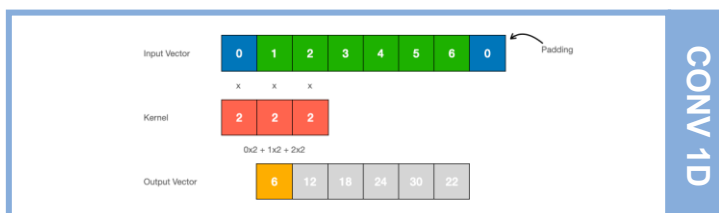
A tensor of rank 4+ representing `activation(conv2d(inputs, kernel) + bias)`.

- filters:** Integer, the dimensionality of the output space (i.e. the number of output filters in the convolution).
- kernel_size:** An integer or tuple/list of 2 integers, specifying the height and width of the 2D convolution window. Can be a single integer to specify the same value for all spatial dimensions.
- strides:** An integer or tuple/list of 2 integers, specifying the strides of the convolution along the height and width. Can be a single integer to specify the same value for all spatial dimensions. Specifying any stride value != 1 is incompatible with specifying any `dilation_rate` value != 1.
- padding:** one of "valid" or "same" (case-insensitive). "valid" means no padding. "same" results in padding with zeros evenly to the left/right or up/down of the input. When `padding="same"` and `strides=1`, the output has the same size as the input.
- data_format:** A string, one of `channels_last` (default) or `channels_first`. The ordering of the dimensions in the inputs. `channels_last` corresponds to inputs with shape $(\text{batch_size}, \text{height}, \text{width}, \text{channels})$ while `channels_first` corresponds to inputs with shape $(\text{batch_size}, \text{channels}, \text{height}, \text{width})$. It defaults to the `image_data_format` value found in your Keras config file at `~/.keras/keras.json`. If you never set it, then it will be `channels_last`.

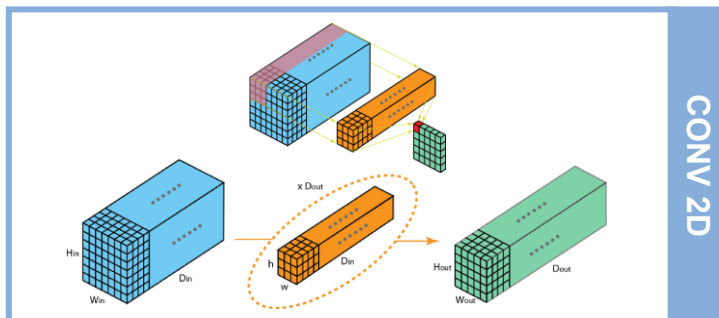
A

21

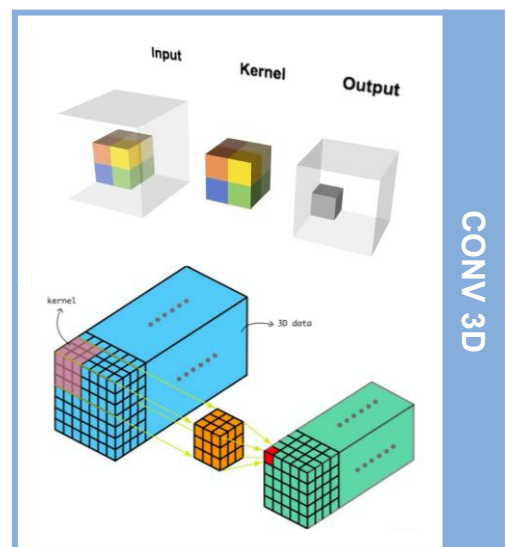
CONV1D, CONV2D, CONV3D



CONV1D



CONV2D

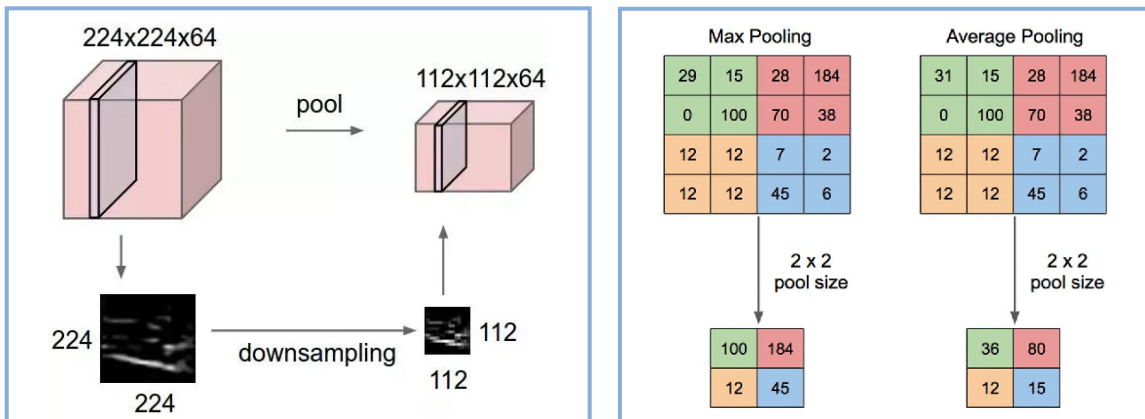


CONV3D

22

A POOLING LAYER (2D)

- Goal of the **pooling layer** is to subsample (shrink) the input image in order to reduce the computational load, the memory usage, and the number of parameters (thereby limiting the risk of overfitting).



23

A POOLING LAYER (2D)

- In CNN, **max pooling** is more common than average pooling.
- Max pooling** also helps introduce some level of invariance to small translations.
- However, **max pooling** is very destructive as many input values are dropped.
- In some applications, invariance is not desirable. For example, semantic segmentation requires equivariance, not invariance.



24

K CODE REVIEW

- MaxPooling1D layer
- MaxPooling2D layer
- MaxPooling3D layer
- AveragePooling1D layer
- AveragePooling2D layer
- AveragePooling3D layer
- GlobalMaxPooling1D layer
- GlobalMaxPooling2D layer
- GlobalMaxPooling3D layer
- GlobalAveragePooling1D layer
- GlobalAveragePooling2D layer
- GlobalAveragePooling3D layer

```
keras.layers.MaxPooling2D(
    pool_size=(2, 2), strides=None, padding="valid", data_format=None, name=None, **kwargs
)
```

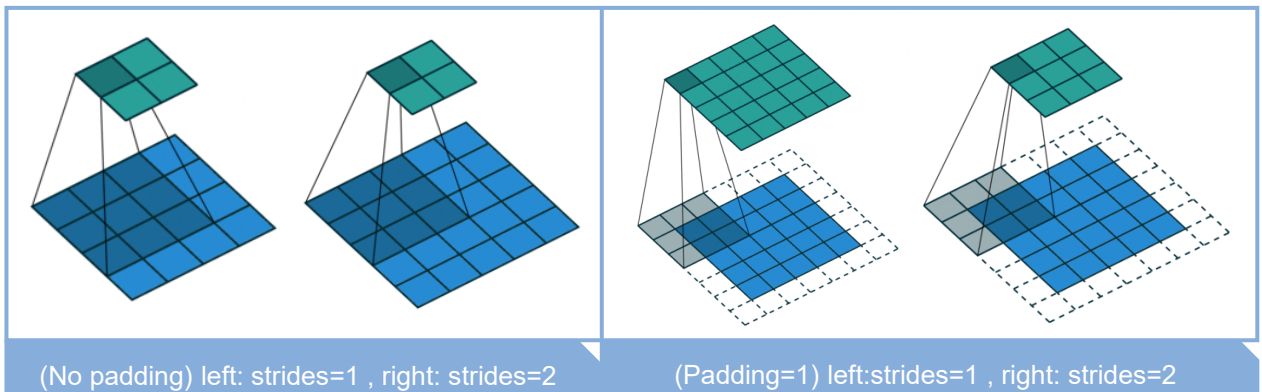
- **pool_size**: integer or tuple of 2 integers, window size over which to take the maximum. (2, 2) will take the max value over a 2x2 pooling window. If only one integer is specified, the same window length will be used for both dimensions.
- **strides**: Integer, tuple of 2 integers, or None. Strides values. Specifies how far the pooling window moves for each pooling step. If None, it will default to `pool_size`.
- **padding**: One of "valid" or "same" (case-insensitive). "valid" means no padding. "same" results in padding evenly to the left/right or up/down of the input such that output has the same height/width dimension as the input.
- **data_format**: A string, one of `channels_last` (default) or `channels_first`. The ordering of the dimensions in the inputs. `channels_last` corresponds to inputs with shape (batch, height, width, channels) while `channels_first` corresponds to inputs with shape (batch, channels, height, width). It defaults to the `image_data_format` value found in your Keras config file at `~/.keras/keras.json`. If you never set it, then it will be "channels_last".

A

25

A STRIDED CONVOLUTION

- Both pooling layer and **strided convolution** (`strides>1`) can be used to encode or reduce the dimensionality of the data.



Animation credit: https://github.com/vdumoulin/conv_arithmetic

26

A STRIDED CONVOLUTION

- Pooling layer vs. **Strided convolutional layer**:
 - Pooling is a fixed operation whereas **(strided) convolution** can be learned.
 - Pooling has no trainable parameter whereas **(strided) convolution** has weights (kernels) and biases to be trained.
 - Pooling is cheaper (faster) in terms of the amount of computation.
 - In many cases, max pooling can be replaced with **strided convolution** without significant change in the accuracy. "Striving for Simplicity: The All Convolutional Net," ICLR2015, <https://arxiv.org/abs/1412.6806>
 - Using max pooling in CNN is historical and appears in most state-of-the-art CNNs. **Strided convolution** is a newer concept.
 - Some researchers said that using **strided convolution** instead of pooling allows gradient signals to flow better during backpropagation. Nevertheless, like most practices in deep learning communities, no one can guarantee that one technique will always outperform the others.

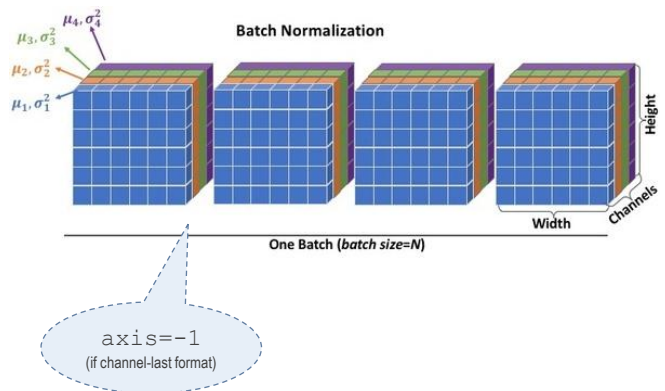
27

A BATCH NORMALIZATION

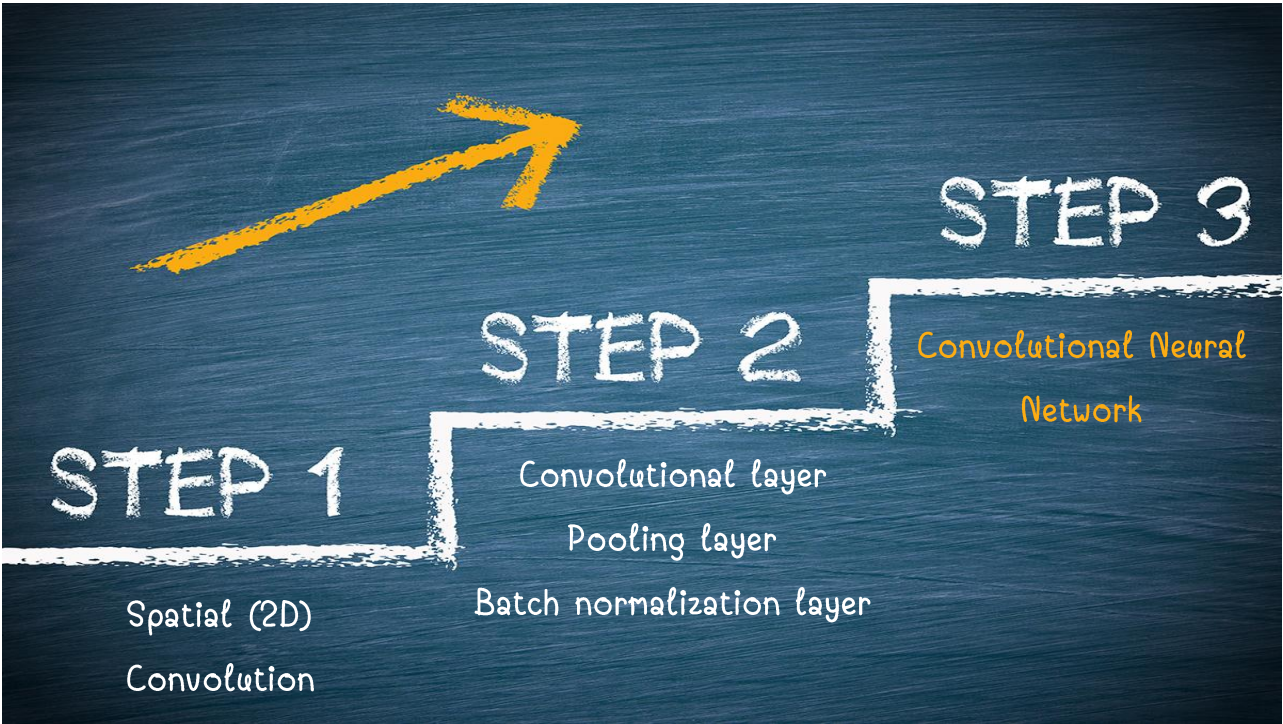
```
keras.layers.BatchNormalization(
    axis=-1,
    momentum=0.99,
    epsilon=0.001,
    center=True,
    scale=True,
    beta_initializer="zeros",
    gamma_initializer="ones",
    moving_mean_initializer="zeros",
    moving_variance_initializer="ones",
    beta_regularizer=None,
    gamma_regularizer=None,
    beta_constraint=None,
    gamma_constraint=None,
    synchronized=False,
    **kwargs
)
```

By default, normalize the last axis by using the means and SDs computed across the other axes

- **axis**: Integer, the axis that should be normalized (typically the features axis). For instance, after a **Conv2D** layer with **data_format="channels_first"**, set **axis=1** in **BatchNormalization**.

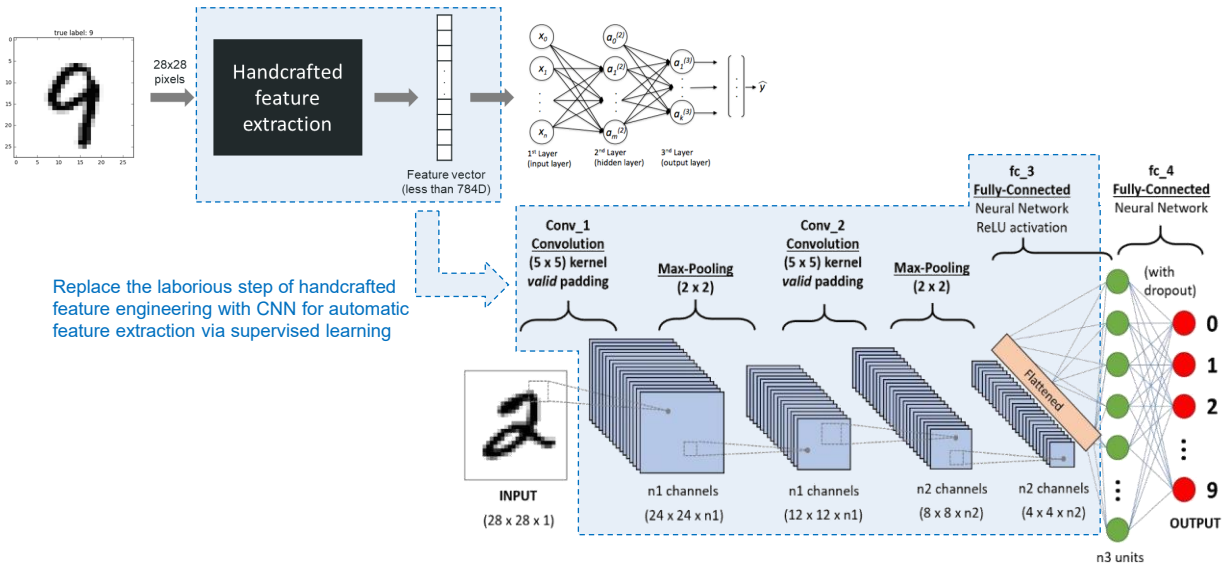


28



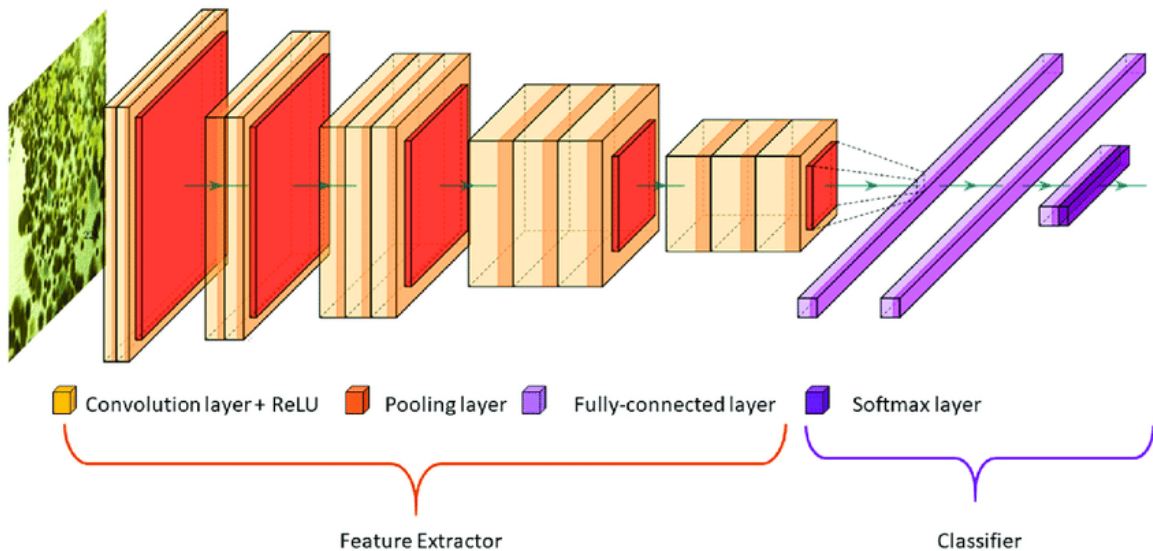
29

CNN for Image Classification



30

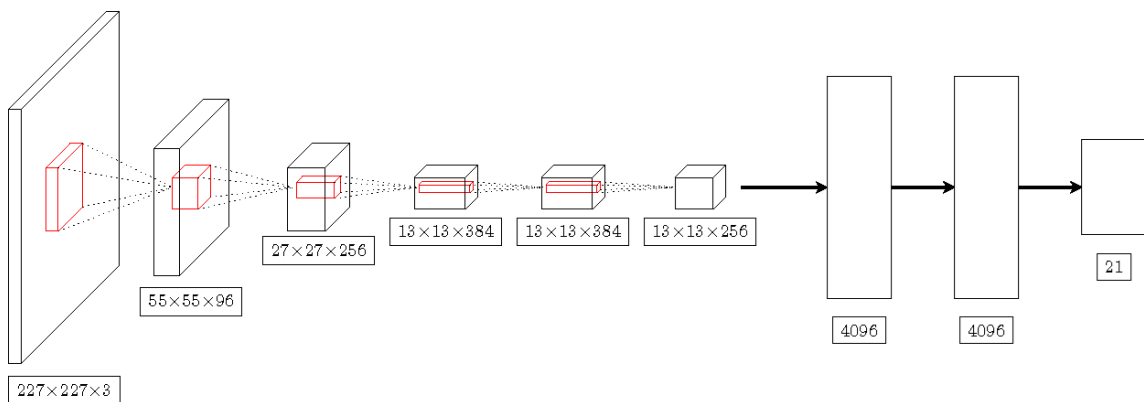
CNN for Image Classification



31

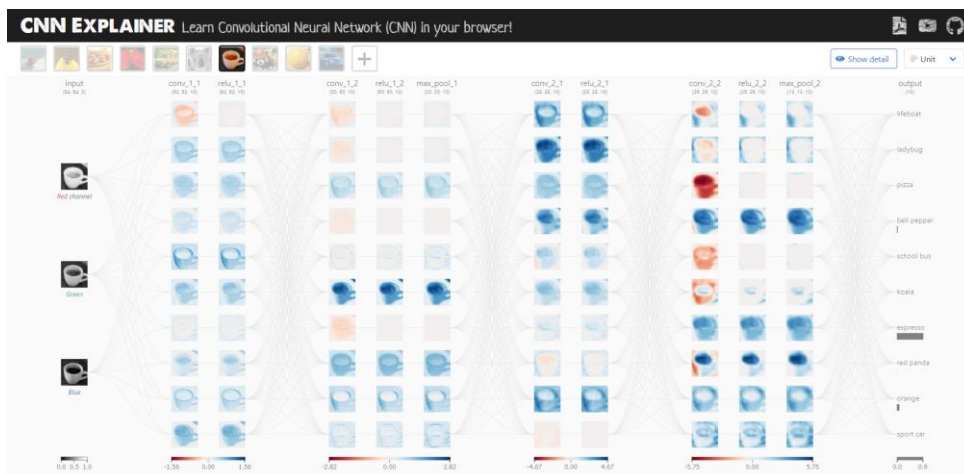
EX1: Convolutional Neural Network (CNN)

- Guess operation(s) being done at each step
- Compute the number of trainable parameters



32

AI CNN EXPLAINER



"CNN Explainer: Learning Convolutional Neural Networks with Interactive Visualization," In IEEE Trans. TVCG, 2020
<https://arxiv.org/abs/2004.15004> | <https://poloclub.github.io/cnn-explainer/>

33



Coding

Combine everything and implement a program with Keras

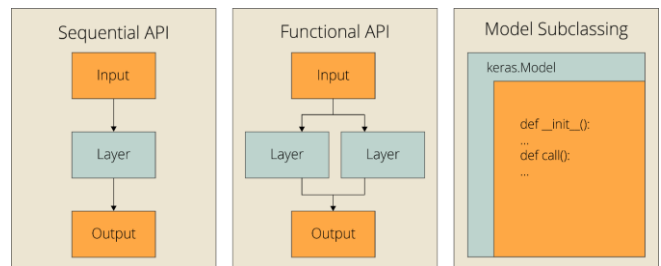
34

CREATING A MODEL IN KERAS

 1. Sequential API `keras.models.Sequential()`

 2. Functional API `keras.models.Model()`

 3. Model Subclassing



35

EX2: Sequential API vs. Functional API

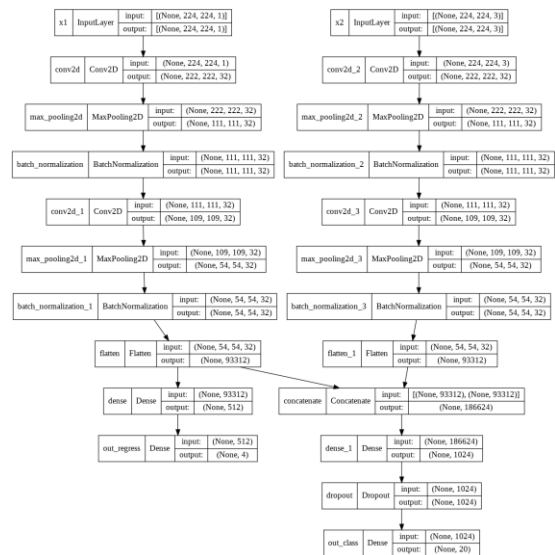
- Sequential model:
 - Input: 3-channel RGB image of 502x502 dimension
 - Conv2D_1 (ReLu): 128 kernels of size 7x7, no padding, stride=1
 - Conv2D_2 (ReLu): 64 kernels of size 5x5, no padding, stride=1
 - MaxPool2D_1: pool size = 2x2, stride=2
 - BatchNormalization
 - Conv2D_3 (ReLu): 32 kernels of size 3x3, padding=same, stride=1
 - Conv2D_4 (ReLu): 16 kernels of size 3x3, padding=same, stride=2
 - Conv2D_5 (ReLu): 16 kernels of size 3x3, no padding, stride=2
 - BatchNormalization
 - Flatten
 - Dense1 (ReLu): 1024 nodes
 - Dropout (drop 50%)
 - Dense2 (ReLu): 1024 nodes
 - Dropout (drop 50%)
 - Output (softmax): 1000 nodes
- Write a program to create this model
- Observe output dimensions and the number of parameters regarding each step

36

EX3: Non-sequential CNN

- Non-sequential model refers to:
 - Multiple inputs and/or outputs
 - Skip connection
 - Branching or splitting
- Further reading: compile with multiple losses (for multiple outputs)
 - `model.compile()`

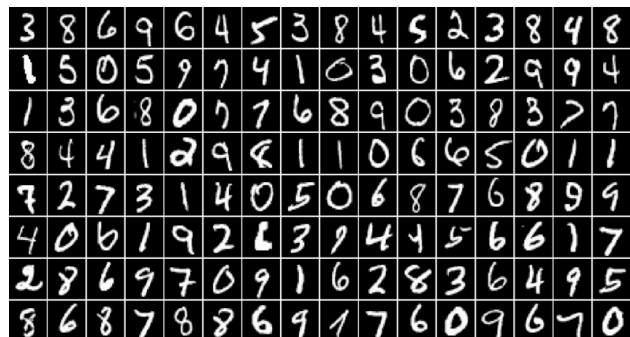
• **optimizer:** String (name of optimizer) or optimizer instance. See `keras.optimizers`.
 • **loss:** Loss function. May be a string (name of loss function), or a `keras.losses.Loss` instance. See `keras.losses`. A loss function is any callable with the signature `loss = fn(y_true, y_pred)`, where `y_true` are the ground truth values, and `y_pred` are the model's predictions. `y_true` should have shape `(batch_size, d0, .. dn)` (except in the case of sparse loss functions such as sparse categorical crossentropy which expects integer arrays of shape `(batch_size, d0, .. dn-1)`). `y_pred` should have shape `(batch_size, d0, .. dn)`. The loss function should return a float tensor.
 • **loss_weights:** Optional list or dictionary specifying scalar coefficients (Python floats) to weight the loss contributions of different model outputs. The loss value that will be minimized by the model will then be the *weighted sum* of all individual losses, weighted by the `loss_weights` coefficients. If a list, it is expected to have a 1:1 mapping to the model's outputs. If a dict, it is expected to map output names (strings) to scalar coefficients.



37

EX4: MNIST Digit Classification

- MNIST dataset for handwritten digit classification
- Design, train, and evaluate one MLP (all layers are Dense layers)
- Design, train, and evaluate one CNN
- Compare results from the two models of yours



38

EX4: MNIST Digit Classification

- **TIPs:**

- Unlike structured data, there often are multiple hidden layers (conv2D layers) when doing feature extraction on a 2D image.
- For faster convergence, don't forget to scale/normalize/standardize each pixel value (originally they were 8-bit unsigned integers ranging between 0 and 255).
- Scaling/normalizing/standardizing pixel values can be done either by explicit NumPy operations or by implicit [Keras preprocessing layers API](#).

```
keras.layers.Normalization(axis=-1, mean=None, variance=None, invert=False, **kwargs)
```

Image preprocessing layers

- Resizing layer
- Rescaling layer
- CenterCrop layer
- AutoContrast layer

Working with preprocessing layers
https://keras.io/api/layers/preprocessing_layers/

https://keras.io/api/layers/preprocessing_layers/

39



40